

JAVASCRIPT ZERO TO HERO SERIES

Day 4: Loops & Control Flow

Mastering `for`, `while`, `do...while`, `break`, `continue` & Nested Loops

 Complete Code Guide

 30 Min Video Lecture

Day 4 Agenda & Learning Roadmap

What you will learn in this session step by step

01. Loop Concept

Why loops matter & roll-call real life analogy.

02. for Loop

Syntax breakdown, live execution trace & common mistakes.

03. while & do...while

Condition-driven loops & key execution differences.

04. Control Flow

break vs **continue** statements in detail.

05. Nested Loops

Multiplication tables & visual star pattern logic.

06. Tasks & Recap

5 live coding challenges & homework assignments.

CORE PROBLEM

Why Do We Need Loops?

Eliminating repetitive manual code

✗ Without Loops (Repetitive & Inefficient)

```
console.log(1);  
console.log(2);  
console.log(3);  
// ... repeating 10 times manually  
console.log(10);
```

✓ Key Insight

- ✓ **Scalability Bottleneck:** Imagine printing numbers from 1 to 1000.
- ✓ **Loop Automation:** Write the instruction ONCE, run it N times.
- ✓ **Clean Code:** Replaces hundreds of lines with 3 compact lines.

Real-World Analogy: Classroom Roll-Call

Understanding loop execution cycles

1. The Repeat Action

Call student name, mark present/absent.

2. The Exit Condition

Are there more students left in class list?

3. Counter Step

Move down to the next roll number on list.

 JavaScript provides 3 core loop types for this pattern: **for**, **while**, and **do...while**.

The **for** Loop Syntax

Used when the exact number of iterations is known in advance

```
for (let i = 1; i ≤ 10; i++) {  
  console.log(i);  
}
```

1. Initialization

let i = 1

Runs ONCE at start. Declares counter variable.

2. Condition

i ≤ 10

Checked BEFORE every step. True = run loop body.

3. Increment

i++

Runs AFTER every step. Updates counter variable.

for Loop Step-by-Step Trace

Tracing counter state changes manually

Step / Iteration	Counter Value (i)	Condition Check (i <= 10)	Console Output	Next Action
Iteration 1	i = 1	1 <= 10 (True)	1	Execute i++ (i becomes 2)
Iteration 2	i = 2	2 <= 10 (True)	2	Execute i++ (i becomes 3)
Iteration 10	i = 10	10 <= 10 (True)	10	Execute i++ (i becomes 11)
Termination	i = 11	11 <= 10 (False)	None	Loop Stops Immediately

Practical **for** Loop Variations

Counting backwards, step skipping & accumulators

1. Countdown

```
for (let i = 10; i ≥ 1; i--) {  
  console.log(i);  
}
```

2. Skip by 2

```
for (let i = 0; i ≤ 20; i += 2) {  
  console.log(i);  
}
```

3. Sum 1 to 100

```
let sum = 0;  
for (let i = 1; i ≤ 100; i++) {  
  sum += i;  
}
```

Common Mistake: Infinite Loop

Wrong counter direction causing crash

```
//  DANGER: i keeps decreasing: 1, 0, -1, -2 ...  
for (let i = 1; i ≤ 10; i--) {  
  console.log(i); // Condition (i ≤ 10) is ALWAYS true!  
}
```

 **Why it happens:** Counter decreases instead of increasing. Condition never turns false, freezing browser tab!

CORRECT APPROACH



Fixed Version: Valid Incrementor

Matching step direction with target condition

```
//  CORRECT: i increases towards exit condition (10)
for (let i = 1; i ≤ 10; i++) {
  console.log(i); // Stops cleanly when i becomes 11
}
```

 **Golden Rule:** Always ensure counter variable progresses TOWARDS false condition exit.

The **while** Loop Syntax

Used when iteration count depends on a dynamic condition

```
let i = 1; // 1. Initialization (Outside)

while (i ≤ 10) { // 2. Condition Check
  console.log(i);
  i++; // 3. Increment (Inside body)
}
```

➡ Entry-Controlled

Evaluates condition FIRST. If initially false, code never executes.

⚙ Manual Counter

Initialization and incrementing are managed separately from loop declaration.

while Loop: Game Lives Simulator

Condition-based repetition in dynamic systems

```
let lives = 3;

while (lives > 0) {
  console.log("Playing game ... Lives left:", lives);
  lives--; // Decrement lives on loss
}

console.log("Game Over!");
```

 Ideal for scenarios where state changes unpredictably during program execution.

✖ Common Mistake: Missing Counter Update

Forgetting increment inside while loop

```
let i = 1;

while (i ≤ 10) {
  console.log(i);
  // ✖ Forgot i++ ! i remains 1 forever
}
```

⚠ Unlike **for** loops, **while** loops do NOT automatically update counters!

CORRECT APPROACH



Fixed Version: Explicit Counter Step

Always updating condition-changing variables

```
let i = 1;

while (i ≤ 10) {
  console.log(i);
  i++; // ✅ Counter updated properly
}
```

✅ Checklist rule: Every while loop MUST have a statement that eventually alters condition state.

LOOP #3

The `do...while` Loop Syntax

Exit-controlled loop guaranteeing at least ONE execution

```
let i = 1;

do {
  console.log(i);
  i++;
} while (i ≤ 10);
```

Guaranteed Run

Code body runs FIRST, then condition is evaluated at the bottom.

Practical Use Case

Prompting user for password/input at least once before validating.

while vs do...while Execution Test

Testing initial FALSE condition (num = 20)

Regular while (Condition Checked First)

```
let num = 20;

while (num ≤ 10) {
  console.log("Runs?");
}

// ❌ Output: Nothing printed!
```

do...while (Body Runs First)

```
let num = 20;

do {
  console.log("Runs once!");
} while (num ≤ 10);

// ✅ Output: "Runs once!"
```

The **break** Statement

Immediately aborts the loop entirely

```
for (let i = 1; i ≤ 10; i++) {  
  if (i === 5) {  
    break; // Aborts loop completely when i equals 5  
  }  
  console.log(i);  
}  
// Output: 1, 2, 3, 4
```

⊘ When break triggers, execution jumps directly to code AFTER loop body.

break: Search Array Pattern

Stopping search once target match is found

```
let numbers = [4, 7, 11, 15, 20, 25];

for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] > 15) {
    console.log("Found target number:", numbers[i]);
    break; // Exit loop early! Saves CPU cycles.
  }
}
```

⚡ Performance Tip: Avoid scanning thousands of extra items once data is located.

The **continue** Statement

Skips current iteration & moves to next step

```
for (let i = 1; i ≤ 10; i++) {  
  if (i === 5) {  
    continue; // Skip printing 5 only  
  }  
  console.log(i);  
}  
  
// Output: 1, 2, 3, 4, 6, 7, 8, 9, 10 (Only 5 skipped)
```

▶ Use **continue** when filtering out specific unwanted items without stopping loop.

break vs continue Comparison

Understanding control flow behavior

❏ break Statement

- Stops the loop **completely**
- Skips all remaining iterations
- Jumps to code after loop
- Analogy: Emergency brake

▶▶ continue Statement

- Skips **current step only**
- Loop continues running next step
- Remains inside loop body
- Analogy: Skipping a song

Nested Loops: Loop Inside a Loop

Outer loop triggers full inner loop cycle

```
for (let i = 1; i ≤ 3; i++) { // Outer Loop
  for (let j = 1; j ≤ 3; j++) { // Inner Loop
    console.log(`i=${i}, j=${j}`);
  }
}
```



For EVERY single iteration of outer counter **i**, inner counter **j** runs completely from start to end.

Nested Loops: Math Tables Example

Generating structured multiplication matrices

```
for (let i = 1; i ≤ 3; i++) {  
  console.log(`--- Table of ${i} ---`);  
  for (let j = 1; j ≤ 5; j++) {  
    console.log(`${i} x ${j} = ${i * j}`);  
  }  
}
```

 Outer loop handles Table Number (1..3); Inner loop handles Multipliers (1..5).

Nested Loops: Star Pattern Building

Classic coding interview challenge

```
for (let i = 1; i ≤ 5; i++) {  
  let pattern = "";  
  for (let j = 1; j ≤ i; j++) {  
    pattern += "* ";  
  }  
  console.log(pattern);  
}
```

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Which Loop to Use, When?

The Golden Rules of loop selection

Scenario / Use Case	Best Tool	Key Advantage
Exact count known in advance	for loop	Initialization, condition & increment on 1 clean line
Dynamic condition, unknown count	while loop	Flexible condition check before each step
Must run at least once regardless	do...while loop	Exit-controlled execution guarantees 1st run
Stop search immediately on match	break	Aborts loop completely to save system resources
Skip specific unwanted values	continue	Bypasses current step & advances to next



Live Practice Tasks

Complete these challenges during recording session

- 1 Write a **for loop** printing even numbers from 1 to 50.
- 2 Write a **while loop** counter printing attempt number until counter reaches 5.
- 3 Use **break** to stop search when finding number 7 in `[2, 5, 7, 9, 11]`.
- 4 Use **continue** to print 1 to 20, skipping numbers divisible by 3.
- ★ **Bonus Challenge:** Write nested loops to print number pyramid pattern (1 / 1 2 / 1 2 3 / 1 2 3 4).

SUMMARY

Day 4 Recap & Homework

Key takeaways to lock in learning

☰ Key Takeaways

- Loops eliminate code duplication & automate tasks.
- **for** = known count; **while** = dynamic conditions.
- **do...while** guarantees at least one execution.
- **break** stops loop; **continue** skips current step.
- Always update counter variables to avoid infinite loops.

🏠 Homework Challenge

1. Complete all 5 live practice tasks.
2. Build a reverse star pattern using nested loops.
3. Share your solution in the course comment section!

UP NEXT: DAY 5

Mastering Functions in JavaScript

Declarations, Expressions, Parameters, Return Values & Default Arguments

Declarations

Writing reusable function blocks.

Parameters

Passing dynamic inputs to functions.

Return Values

Extracting computed outputs.

Thank You! Like, Share & Subscribe | Allah Hafiz!